Operating Systems

Lecturer
Dr. Fatma Ahmed Hossam EL-Din

Chapter 4: Threads

Chapter 4: Threads

- Introduction
- Multicore Programming
- Multithreading Models
- Benefits of Threads
- Thread Libraries

Introduction

- A Thread is a basic unit utilization; it comprises a thread ID, a program counter, a register set, and a stack
- It shares with other threads belonging to the same process its code section, data section, and other operating-system, such as open files and operating-system resources, sch as open files and signals
- To introduce the notion of a thread—a fundamental unit of CPU utilization that forms the basis of multithreaded computer systems

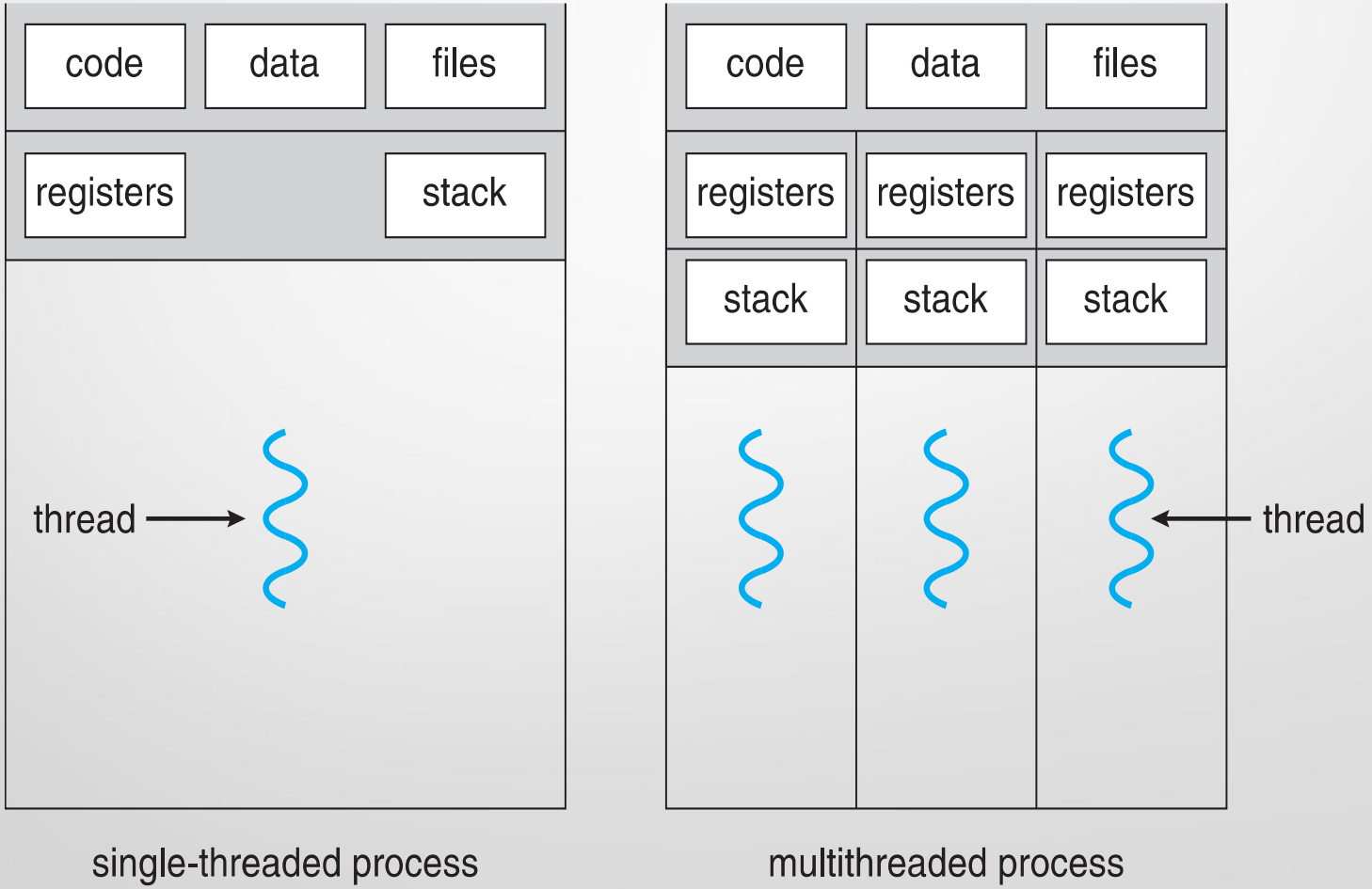
Introduction, Motivation

- Let`s say, for example, a program is not capable of drawing picture while reading keystrokes. The program must give its full attention to the keyboard input lacking the ability to handle more than one event at a time.
- The ideal solution to this problem is the seamless execution of two or more sections of a program at the same time. Threads allows us to do this.
- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
  - Update display
  - Fetch data
  - Spell checking
  - Answer a network request
- Process creation is heavy-weight while thread creation is light-weight

Kernels are generally multithreaded

Introduction

Single and Multithreaded Processes

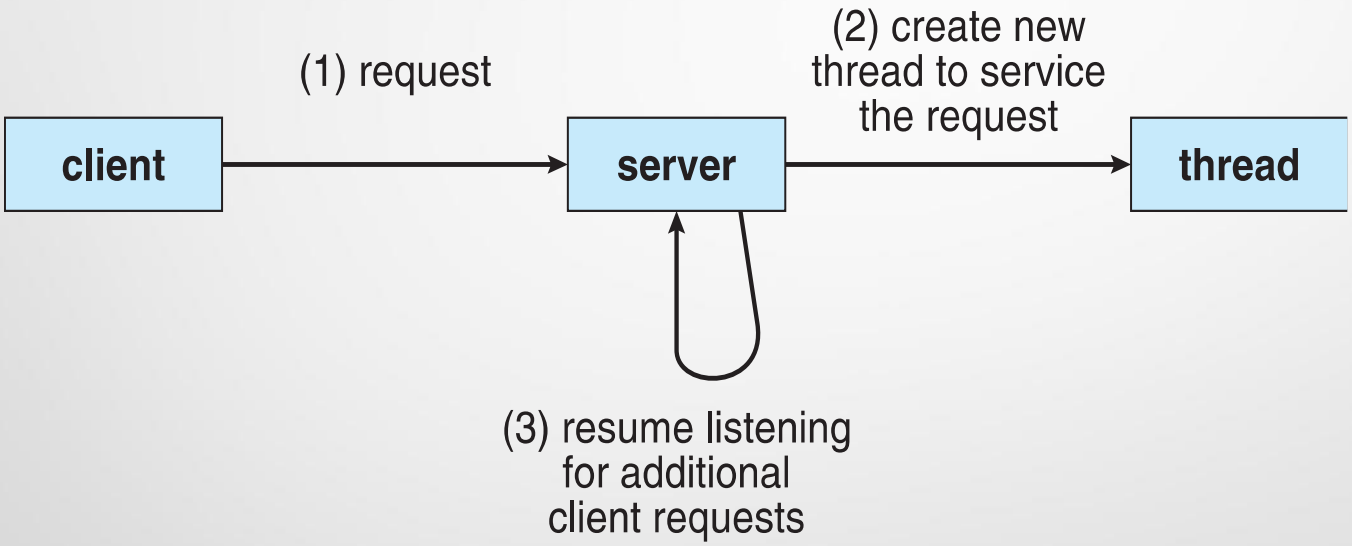


Introduction

- Most software applications that run on modern computers are **multithreaded**
- An application typically is implemented as a separate process with several threads of control.
  - A Web browser might have one thread display images or text while another thread retrieves data from the network, for example
  - A Word processor may have a thread for displaying graphics, another thread for responding to keystrokes from the user, and a third thread for performing spelling and grammar checking in the background.

Introduction

Multithreaded Server Architecture



Introduction

- Finally, most operating-systems kernels are now multithreaded. Several threads operate in the kernel, and each thread performs task, such as managing devices, managing memory, or interrupt handling.

Multicore Programming

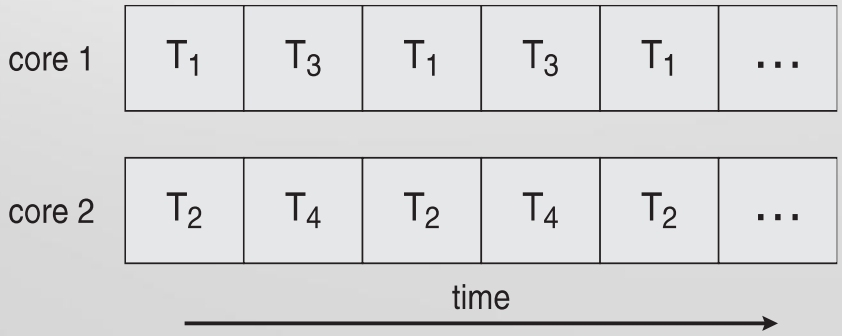
- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
  - **Dividing activities**
  - **Balance**
  - **Data splitting**
  - **Data dependency**
  - **Testing and debugging**
- ***Parallelism*** implies a system can perform more than one task simultaneously
- ***Concurrency*** supports more than one task making progress
  - Single processor / core, scheduler providing concurrency

Multicore Programing
Concurrency vs. Parallelism

- Multithreaded programming provides a mechanism for more efficient use of these multicore or multiprocessor system

- **Concurrent execution on single-core system:**

- **Parallelism on a multi-core system:**



Multicore Programming (Cont.)

- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple core
  - **Task parallelism** – distributing threads across cores, each thread performing unique operation

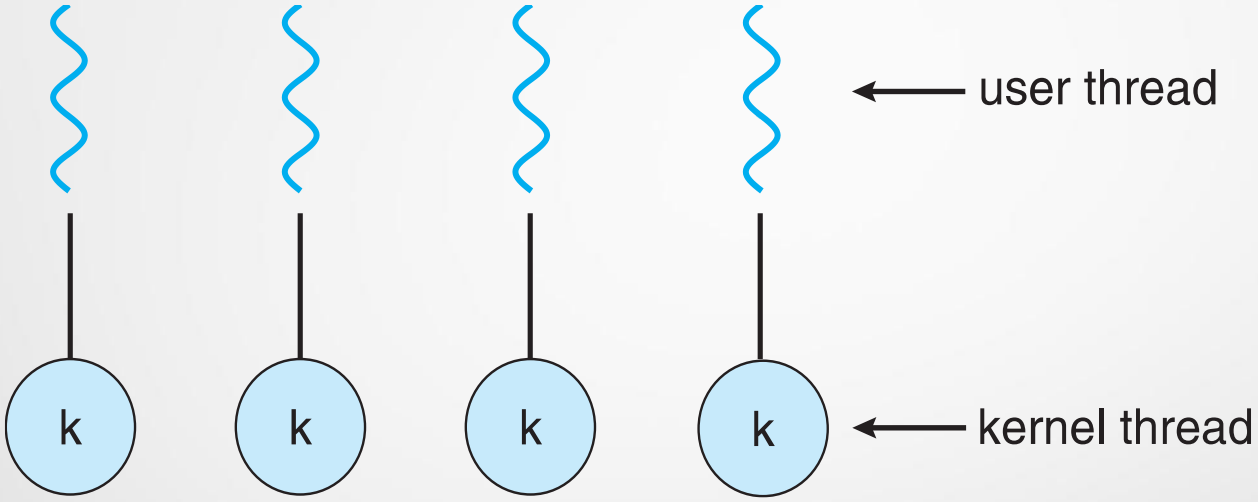
Multithreading Models
User Threads and Kernel Threads

- Support for threads may be provided either at the user level, **for user threads**, or by the kernel, for **kernel threads**
- User Threads ae supported above the kernel and are managed without kernel support, whereas kernel threads are supported and managed directly by the operating system
- **User threads** - management done by user-level threads library
  - Three primary thread libraries:
    - POSIX **Pthreads,** Windows threads, Java threads
- **Kernel threads** - Supported by the Kernel
  - Examples – virtually all general-purpose operating systems, including: Windows, Solaris, Linux, Tru64 UNIX, Mac OS X

Multithreading Models

- Ultimately, a relationship must exist between user threads and kernel threads
- Four Common models:
  - One-to-One model
  - Many-to-One
  - Many-to-Many
  - Two-level Model

One-to-One



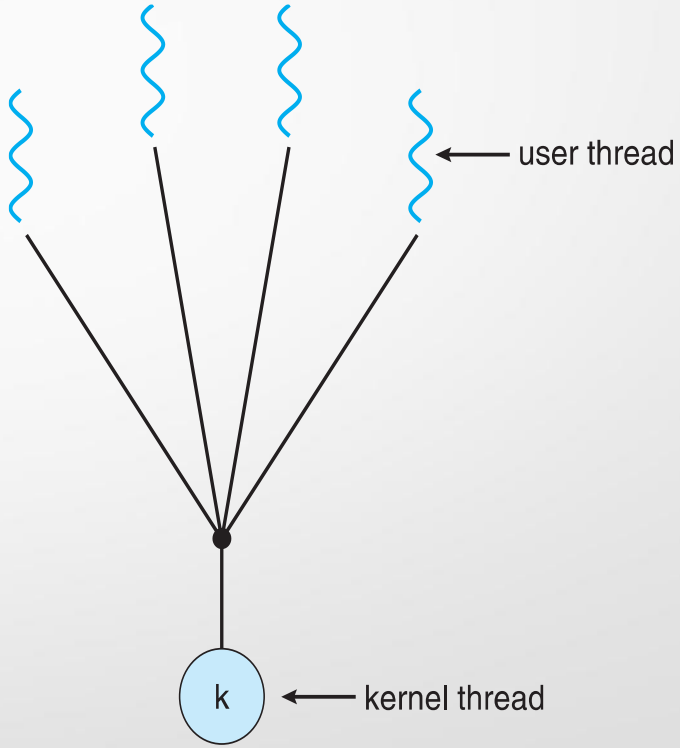
- The one-to-one model maps each user thread to kernel thread. It also allows multiple threads to run in parallel on multiprocessors

One-to-One

- The only **drawback** to this model is that **creating a user thread requires creating the corresponding kernel thread**. Because the overhead of creating kernel threads can burden the performance of an application, most implementations of this model restrict the number of threads supported by the system. Linex, along with the family of windows operating systems, implement the one-to-one model

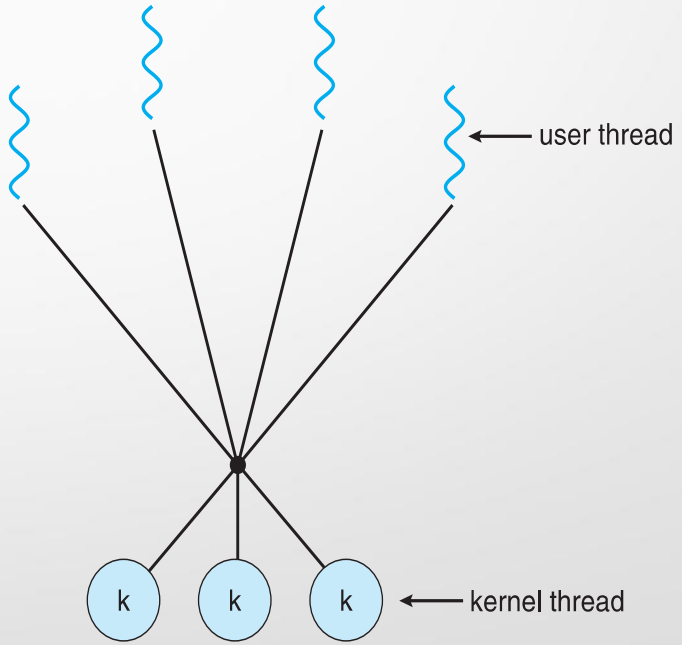
Many-to-One

- Many user-level threads mapped to single kernel thread
- Few systems currently use this model
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**



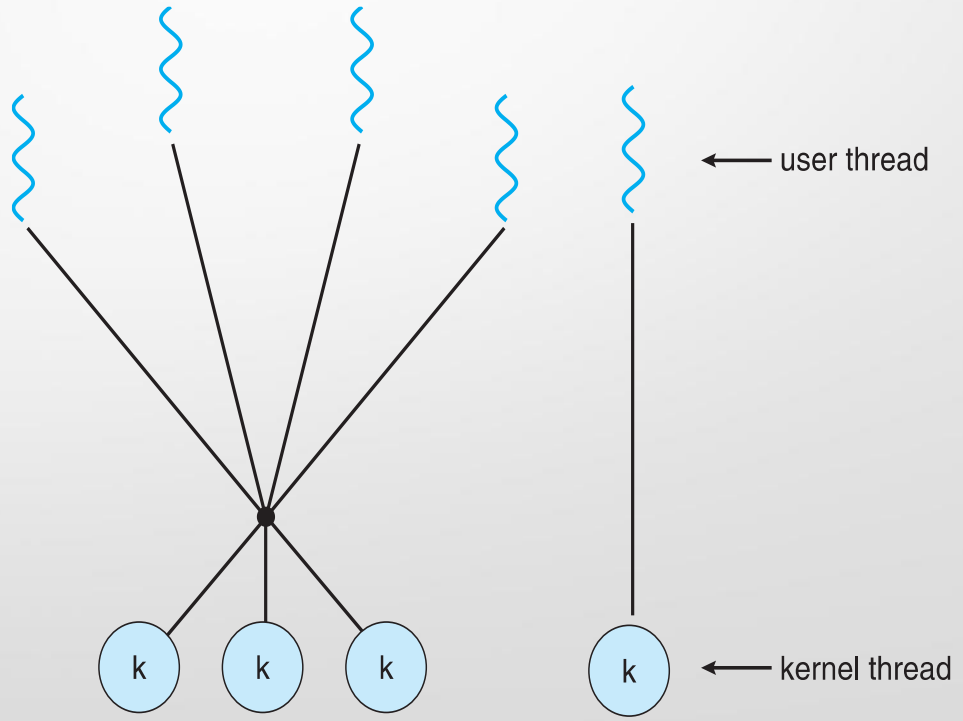
Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Allows the developer to create as many user threads as she wishes, it does not result in true concurrency, because the kernel can schedule only thread at a time.



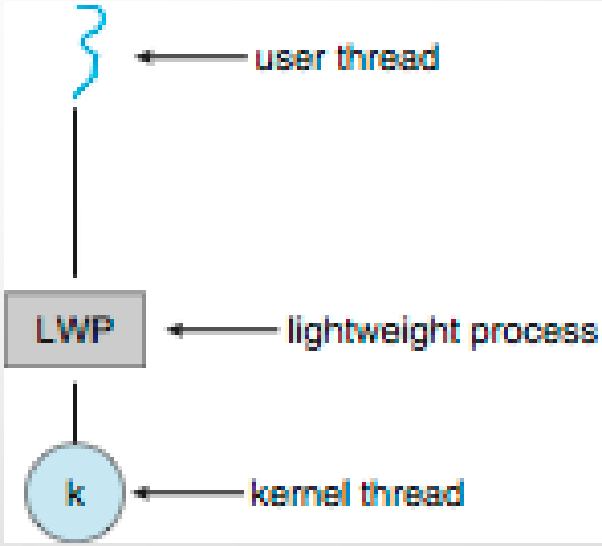
Two-level Model

- Similar to Many-to-Many, except that it allows a user thread to be **bound** to kernel thread
- Examples
  - IRIX
  - HP-UX
  - Tru64 UNIX
  - Solaris 8 and earlier



Scheduler Activations

- Both M:M and Two-level models require communication to maintain the appropriate number of kernel threads allocated to the application
- Typically use an intermediate data structure between user and kernel threads – **lightweight process** (**LWP**)
  - The LWP appears to be a virtual processor on which the application can schedule a user thread to run, to the user-thread library. Each Light Weight Process is attached to a kernel thread, and it is kernel threads that the operating system schedules to run on physical processors.



This communication allows an application to maintain the correct number kernel threads

Thread Libraries

- **Thread library** provides programmer with API for creating and managing threads
- Three main thread libraries are in use today:
  - POSIX Pthreads
  - Windows, and Java
- Two primary ways of implementing
  - Library entirely in user space
  - Kernel-level library supported by the OS

Operating System Examples

- Windows Threads

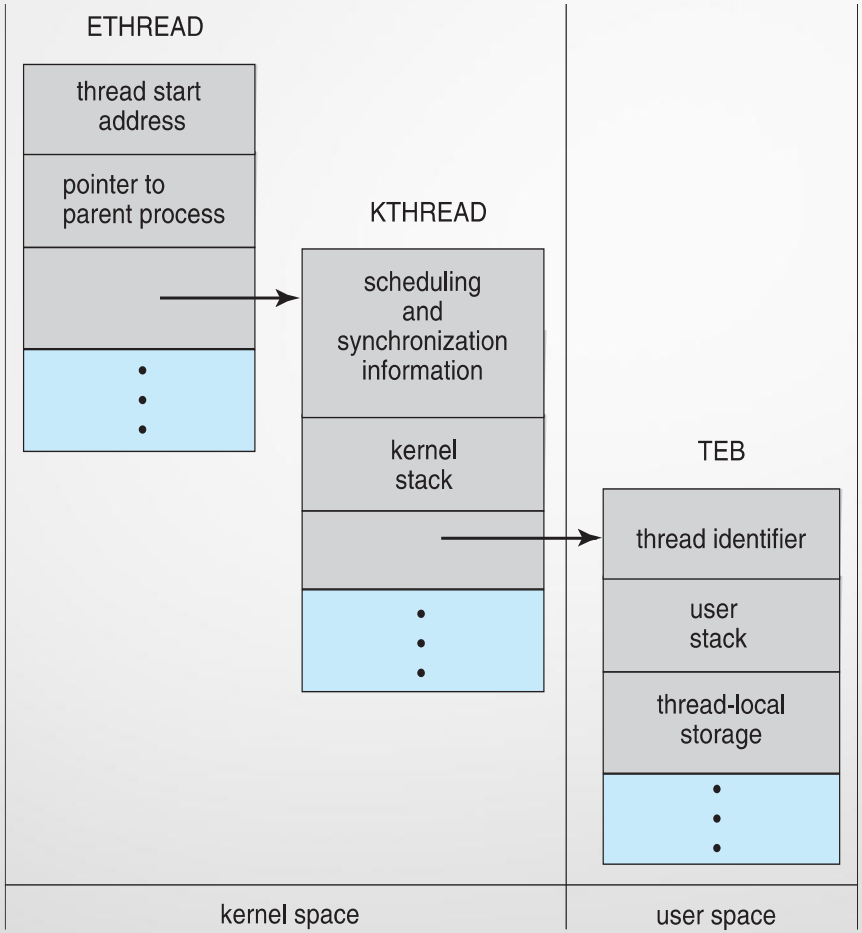
Windows Threads

- Windows implements the Windows API – primary API for Win 98, Win NT, Win 2000, Win XP, and Win 7
- Implements the one-to-one mapping, kernel-level
- Each thread contains
  - A thread id
  - Register set representing state of processor
  - Separate user and kernel stacks for when thread runs in user mode or kernel mode
  - Private data storage area used by run-time libraries and dynamic link libraries (DLLs)
- The register set, stacks, and private storage area are known as the **context** of the thread

Windows Threads (Cont.)

- The primary data structures of a thread include:
  - ETHREAD (executive thread block) – includes pointer to process to which thread belongs and to KTHREAD, in kernel space
  - KTHREAD (kernel thread block) – scheduling and synchronization info, kernel-mode stack, pointer to TEB, in kernel space
  - TEB (thread environment block) – thread id, user-mode stack, thread-local storage, in user space

Windows Threads Data Structures



Benefits

- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** process can take advantage of multiprocessor architectures

End of Chapter 4
Thank you